```
import * as React from "react"

interface SeatProps {
  seatNumber: number
}

const Seat = (props: SeatProps): React.ReactElement => {
  return (
    <td>
      <span
        className="p-4 m-2 border-black border-4 button
                hover:bg-blue-300 text-lg">
        {props.seatNumber + 1}
      </span>
    </td>
  )
}

export default Seat
```

The core of this file is the actual component, which is the function named `Seat`. You'll notice that the syntax inside the `Seat` function does not look like standard JavaScript or TypeScript. Instead we have this weird `<td>` there, and it looks like we dropped HTML directly into our TypeScript file, and somehow it's all working.

That syntax is JSX. JSX is an extension to JavaScript that allows HTML-like elements inside angle brackets to be included in JavaScript or TypeScript code. React uses JSX to define HTML tags that are part of our React components, and also as a way of calling React components from other React components. The React JSX parser converts our `.jsx` and `.tsx` files to plain JavaScript or TypeScript. This conversion has already been set up for us by esbuild using a mechanism we'll look at in more detail in Chapter 7, *Bundling*.

Functional vs. Class Components

